

LAPORAN PRAKTIKUM PEKAN 4

PEMOGRAMAN BERORIENTASI OBJEK



Disusun oleh:

ANNAFI AL GHIFARI

2511533013

Dosen pengampu

Nurfiah, S.ST, M.Kom.

Asisten Praktikum

Dawi Syauqi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

DAFTAR ISI

PROGRAM.....	3
KESIMPULAN	18

PROGRAM

2.2 Program Main.java

2.2.1 Kode Program Main.java

```
package pekan4;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;

        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {

            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("7. Simulasi Akhir Bulan (Khusus Tabungan)");
            System.out.println("8. Simulasi Akhir Tahun (Khusus Tabungan)"); //
            Menu Baru

            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {

                case 1:
```

```

        System.out.print("Masukkan No Rekening: ");
        String no = input.nextLine();

        System.out.print("Masukkan Nama Pemilik: ");
        String nama = input.nextLine();

        System.out.print("Masukkan Saldo Awal: ");

        if (!input.hasNextDouble()) {
            System.out.println("Error: Saldo awal harus berupa
angka!");
            input.next();
            break;
        }

        double saldoAwal = input.nextDouble();
        input.nextLine();

        System.out.print("Masukkan PIN: ");
        String pinAwal = input.nextLine();

        System.out.println("Pilih Produk: 1. Tabungan Umum | 2.
Giro Bisnis");

        System.out.print("Pilihan: ");
        int produk = input.nextInt();
        input.nextLine();

        Rekening rekeningBaru = null;

        if (produk == 1) {
            System.out.print("Masukkan Suku Bunga (%): ");
            double sukuBunga = input.nextDouble();
            input.nextLine();
            rekeningBaru = new RekeningTabungan(no, nama,
saldoAwal, pinAwal, sukuBunga);
        } else if (produk == 2) {
            System.out.print("Masukkan Batas Overdraft: ");
            double batasOverdraft = input.nextDouble();
            input.nextLine();
            rekeningBaru = new RekeningGiro(no, nama, saldoAwal,
pinAwal, batasOverdraft);
        } else {

```

```

        System.out.println("Produk tidak valid, rekening tidak
dibuat.");
        break;
    }

    // Upcasting: RekeningTabungan / RekeningGiro otomatis
    dikenali sebagai Rekening
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal setor: ");
            if (input.hasNextDouble()) {
                double setor = input.nextDouble();
                akunAktif.setor(setor);
            } else {
                System.out.println("Error: Nominal harus berupa
angka!");
                input.next();
            }
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal tarik: ");
            if (input.hasNextDouble()) {
                double tarik = input.nextDouble();
                akunAktif.tarik(tarik);
            } else {
                System.out.println("Error: Nominal harus berupa
angka!");
                input.next();
            }
        }
        break;

```

```

        case 4:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka
rekening!");
            } else {
                akunAktif.cetakInfo();
            }
            break;

        case 5:
            System.out.print("Masukkan nomor rekening: ");
            String nomorCari = input.nextLine();
            boolean ditemukan = false;

            for (Rekening rekening : daftarRekening) {
                if (rekening.getNomorRekening().equals(nomorCari)) {
                    akunAktif = rekening;
                    ditemukan = true;
                    System.out.println("Berhasil mengganti akun ke
rekening " + akunAktif.getNamaPemilik());
                    break;
                }
            }

            if (!ditemukan) {
                System.out.println("Rekening tidak ditemukan!");
            }
            break;

        case 7:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum memiliki rekening
aktif!");
            } else if (akunAktif instanceof RekeningTabungan) {
                // Downcasting: aman karena sudah dicek dengan
instanceof

                RekeningTabungan tab = (RekeningTabungan) akunAktif;
                tab.tambahBungaAkhirBulan();
            } else {
                System.out.println("Gagal: Fitur bunga akhir bulan
hanya berlaku untuk Rekening Tabungan.");
            }
            break;

        case 8:

```

```

        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki rekening
aktif!");
        } else if (akunAktif instanceof RekeningTabungan) {
            RekeningTabungan tab = (RekeningTabungan) akunAktif;
            tab.tambahBungaAkhirTahun();
        } else {
            System.out.println("Gagal: Fitur bunga akhir tahun
hanya berlaku untuk Rekening Tabungan.");
        }
        break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima Kasih!");
        break;

    default:
        System.out.println("Pilihan menu tidak tersedia!");
        break;
    }
}
input.close();
}
}

```

2.2.2 Penjelasan

- ❑ Menampilkan menu interaktif utama menggunakan *looping* while dan struktur kontrol switch-case.
- ❑ Mengelola sekumpulan akun rekening menggunakan koleksi `ArrayList<Rekening>`.
- ❑ **Upcasting:** Saat pembuatan rekening baru (opsi 1), objek `RekeningTabungan` atau `RekeningGiro` dimasukkan ke dalam daftar ber tipe `Rekening`.
- ❑ **Downcasting & instanceof:** Pada simulasi akhir bulan (opsi 7) dan akhir tahun (opsi 8), program memeriksa tipe objek menggunakan keyword `instanceof` untuk memastikan akun aktif adalah `RekeningTabungan` sebelum melakukan *casting* ke tipe spesifik tersebut.

2.3 Program Rekening.java

2.3.1 Kode Program Rekening.java

```
package pekan4;

import java.util.ArrayList;

public class Rekening {
    private String nomorRekening;
    private String namaPemilik;
    private String pin;

    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN
default 123456");
            this.pin = "123456";
        }
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat.");
    }

    public String getNomorRekening() {
        return nomorRekening;
    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    public double getSaldo() {
        return saldo;
    }
}
```

```

public ArrayList<Transaksi> getRiwayatTransaksi() {
    return riwayatTransaksi;
}

public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}

public boolean setor(double nominal) {
    if (nominal <= 0) {
        System.out.println("Nominal setor harus lebih dari 0.");
        return false;
    }
    this.saldo += nominal;
    String id = "TRX" + (riwayatTransaksi.size() + 1);
    riwayatTransaksi.add(new Transaksi(id, "Setor", nominal));
    System.out.println("Setor Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
    return true;
}

public boolean tarik(double nominal) {
    if (nominal <= 0) {
        System.out.println("Nominal tarik harus lebih dari 0.");
        return false;
    }
    if (nominal > saldo) {
        System.out.println("Saldo tidak cukup.");
        return false;
    }
    this.saldo -= nominal;
    String id = "TRX" + (riwayatTransaksi.size() + 1);
    riwayatTransaksi.add(new Transaksi(id, "Tarik", nominal));
    System.out.println("Tarik Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
    return true;
}

public void cetakInfo() {
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Pemilik      : " + namaPemilik);
    System.out.println("Saldo        : Rp" + saldo);
}
}

```

2.3.2 Penjelasan

- nomorRekening & namaPemilik: Menyimpan identitas pemilik rekening (akses private).
- pin: Menyimpan PIN 6 digit untuk keamanan otentikasi. Jika PIN tidak 6 digit, otomatis diset ke nilai *default* 123456.
- saldo: Menyimpan jumlah saldo (akses protected agar dapat diakses langsung oleh kelas turunannya).
- riwayatTransaksi: Daftar berjenis ArrayList<Transaksi> untuk mencatat seluruh riwayat aktivitas transaksi.
- setor(double nominal): Menambahkan saldo dan mencatat transaksi ke riwayat.
- tarik(double nominal): Mengurangi saldo jika nominal valid dan saldo mencukupi.
- otentikasi(String inputPin): Memvalidasi kesesuaian PIN.
- cetakInfo(): Menampilkan informasi nomor rekening, nama pemilik, dan saldo saat ini.

2.4 Program Transaksi.java

2.4.1 Kode Program Transaksi.java

```
package pekan4;

public class Transaksi {

    private String idTransaksi;
    private String jenisTransaksi;
    private double nominal;

    public Transaksi(String idTransaksi, String jenisTransaksi, double nominal)
    {
        this.idTransaksi = idTransaksi;
        this.jenisTransaksi = jenisTransaksi;
        this.nominal = nominal;
    }

    @Override
```

```

    public String toString() {
        return "[" + idTransaksi + "] " + jenisTransaksi + " sebesar Rp" +
nominal;
    }
}

```

2.4.2 Penjelasan

- idTransaksi: Kode unik transaksi.
- jenisTransaksi: Deskripsi tipe transaksi (misal: "Setor", "Tarik Giro", "Bunga Bulanan").
- nominal: Jumlah uang dalam transaksi tersebut.
- toString() (*Overriding*): Mengubah format tampilan objek Transaksi menjadi teks berformat rapi, contoh: [TRX1] Setor sebesar Rp500000.0.

2.5 Program RekeningGiro

2.5.1 Kode Program RekeningGiro.java

```

package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String
pinAwal, double batasOverdraft) {
        // Memanggil konstruktor dari superclass (Rekening)
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    public double getBatasOverdraft() {
        return batasOverdraft;
    }

    // Menimpa (override) method tarik dari class induk khusus untuk Giro
    @Override
    public boolean tarik(double nominal) {
        if (nominal <= 0) {
            System.out.println("Nominal tarik harus lebih dari 0.");

```

```

        return false;
    }

    // Logika Overdraft: Boleh tarik melebihi saldo,
    // asalkan nominal tidak melebihi (saldo + batasOverdraft)
    if (nominal > (saldo + batasOverdraft)) {
        System.out.println("Gagal: Saldo dan limit overdraft tidak
cukup!");
        return false;
    }

    this.saldo -= nominal;

    String id = "TRX" + (riwayatTransaksi.size() + 1);
    riwayatTransaksi.add(new Transaksi(id, "Tarik Giro", nominal));
    System.out.println("Tarik Rp" + nominal + " berhasil. Saldo sekarang:
Rp" + saldo);
    return true;
}
}

```

2.5.2 Penjelasan

- batasOverdraft: Batas maksimum pinjaman/dana cadangan yang diizinkan untuk ditarik ketika saldo utama habis.
- tarik(double nominal) (Polimorfisme / *Method Overriding*): Menimpa logika pencairan pada kelas induk. Penarikan tetap diizinkan meskipun melebihi jumlah saldo, selama total penarikan tidak melebihi saldo + batasOverdraft. Jika berhasil, saldo bisa bernilai negatif hingga batas *overdraft* yang ditentukan.

2.6 Program RekeningTabungan

2.6.1 Kode Program RekeningGiro.java

```

package pekan4;

public class RekeningTabungan extends Rekening {
    private double sukuBunga;

```

```

    public RekeningTabungan(String nomor, String nama, double saldoAwal, String
pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Bulanan",
nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp"
+ nominalBunga);
    }

    public void tambahBungaAkhirTahun() {
        if (saldo > 10000000) {
            double nominalBunga = saldo * (sukuBunga / 100);
            saldo += nominalBunga;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Akhir Tahun",
nominalBunga));

            System.out.println("Selamat! Anda mendapat bonus bunga akhir tahun
" + sukuBunga + "%: Rp" + nominalBunga);
            System.out.println("Saldo Anda sekarang: Rp" + saldo);
        } else {
            System.out.println("Gagal: Saldo Anda (Rp" + saldo + ") tidak
memenuhi syarat minimum > Rp10.000.000 untuk bunga akhir tahun.");
        }
    }
}

```

2.6.2 Penjelasan

- sukuBunga: Menyimpan persentase bunga bulanan/tahunan.
- tambahBungaAkhirBulan(): Menghitung bunga berdasarkan saldo saat ini dan langsung menambahkan nominal bunga tersebut ke saldo serta riwayat transaksi.

- tambahBungaAkhirTahun(): Memberikan bonus bunga akhir tahun khusus bagi nasabah yang memiliki saldo di atas Rp10.000.000. Jika saldo tidak mencukupi syarat, penambahan bunga akan ditolak.

2.7 Output Challenge

Simulasi Akhir Tahun tidak akan bisa kalau saldo dibawah 10JT

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 8
Gagal: Saldo Anda (Rp50000.0) tidak memenuhi syarat minimum > Rp10.000.000 untuk bunga akhir tahun.
```

Simulasi Akhir Tahun terjadi karena saldo diatas 10JT

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 8
Selamat! Anda mendapat bonus bunga akhir tahun 4.0%: Rp402000.0
Saldo Anda sekarang: Rp1.0452E7
```

2.8 Output Tugas

- 1 Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: nafi
Masukkan Saldo Awal: 50000
Masukkan PIN: 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 1
Masukkan Suku Bunga (%): 4
Rekening atas nama nafi berhasil dibuat.
```

- 2 Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.

```
Masukkan Suku Bunga (%): 4
Rekening atas nama nafi berhasil dibuat.
```

- 3 Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.

```
Pilih menu: 1
Masukkan No Rekening: 456
Masukkan Nama Pemilik: ghifari
Masukkan Saldo Awal: 50000
Masukkan PIN: 123456
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 55000
Rekening atas nama ghifari berhasil dibuat.
```

- 4 Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

```
5         daftarRekening.add(rekeningBaru);
6         akunAktif = rekeningBaru;
7         break;
```

- 5 Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Simulasi Akhir Tahun (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Bunga 4.0% berhasil ditambahkan: Rp418080.0
```

- 6 Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator *instanceof*.

```
7     else if (akunAktif instanceof RekeningTabungan) {
8         RekeningTabungan tab = (RekeningTabungan) akunAktif;
9         tab.tambahBungaAkhirBulan();
10    } else {
```

```
11         System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku  
    untuk Rekening Tabungan.");
```

- 7 Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

```
8     public void tambahBungaAkhirTahun() {  
9         if (saldo > 10000000) {  
10             double nominalBunga = saldo * (sukuBunga / 100);  
11             saldo += nominalBunga;  
12  
13             String idTrx = "TRX-T-" + System.currentTimeMillis();  
14             riwayatTransaksi.add(new Transaksi(idTrx, "Bunga Akhir Tahun",  
    nominalBunga));  
15  
16             System.out.println("Selamat! Anda mendapat bonus bunga akhir  
    tahun " + sukuBunga + "%: Rp" + nominalBunga);  
17             System.out.println("Saldo Anda sekarang: Rp" + saldo);  
18         } else {  
19             System.out.println("Gagal: Saldo Anda (Rp" + saldo + ") tidak  
    memenuhi syarat minimum > Rp10.000.000 untuk bunga akhir tahun.");  
20         }  
21     }
```

- 8 Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
7. Simulasi Akhir Bulan (Khusus Tabungan)  
8. Simulasi Akhir Tahun (Khusus Tabungan)  
0. Keluar  
Pilih menu: 7  
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.
```

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan pada studi kasus sistem perbankan sederhana, dapat disimpulkan beberapa hal sebagai berikut:

1. Penerapan Konsep Pemrograman Berbasis Objek (PBO):

- **Inheritance (Pewarisan):** Kelas RekeningTabungan dan RekeningGiro berhasil mengimplementasikan sifat pewarisan dari *superclass* Rekening, sehingga atribut dasar (seperti nomorRekening, namaPemilik, dan saldo) serta metode operasional dapat diturunkan dan dikembangkan secara efisien tanpa membuat kode berulang (*code redundancy*).
- **Polymorphism & Method Overriding:** Terjadi penimpaan (*override*) metode tarik() pada kelas RekeningGiro untuk memberikan perilaku spesifik berupa aturan fasilitas penarikan melebihi saldo (*overdraft*).
- **Upcasting & Downcasting:**
 - *Upcasting* diterapkan saat menyimpan berbagai objek produk turunan (RekeningTabungan atau RekeningGiro) ke dalam satu wadah koleksi sejenis (ArrayList<Rekening>).
 - *Downcasting* dikombinasikan dengan pemrosesan pengecekan tipe objek menggunakan operator instanceof pada menu simulasi akhir bulan (case 7), memastikan bahwa metode khusus tambahBungaAkhirBulan() hanya dipanggil secara aman oleh objek yang terbukti berjenis RekeningTabungan.

2. Pengembangan Antarmuka Interaktif (CLI):

- Modifikasi antarmuka CLI pada Main.java berhasil mendukung pembuatan multi-produk secara dinamis berdasarkan masukan pengguna, serta mampu menangani pembatasan akses fitur khusus produk secara opsional dan aman.